Kathmandu University

Department of Computer Science and Engineering

Dhulikhel, Kavre

Lab Report 2

[Course Code: COMP 307]

Submitted by:
Jatin Madhikarmi
Roll No. 32

Submitted to:
Ms. Rabina Shrestha
Department of Computer Science and Engineering

1 Introduction

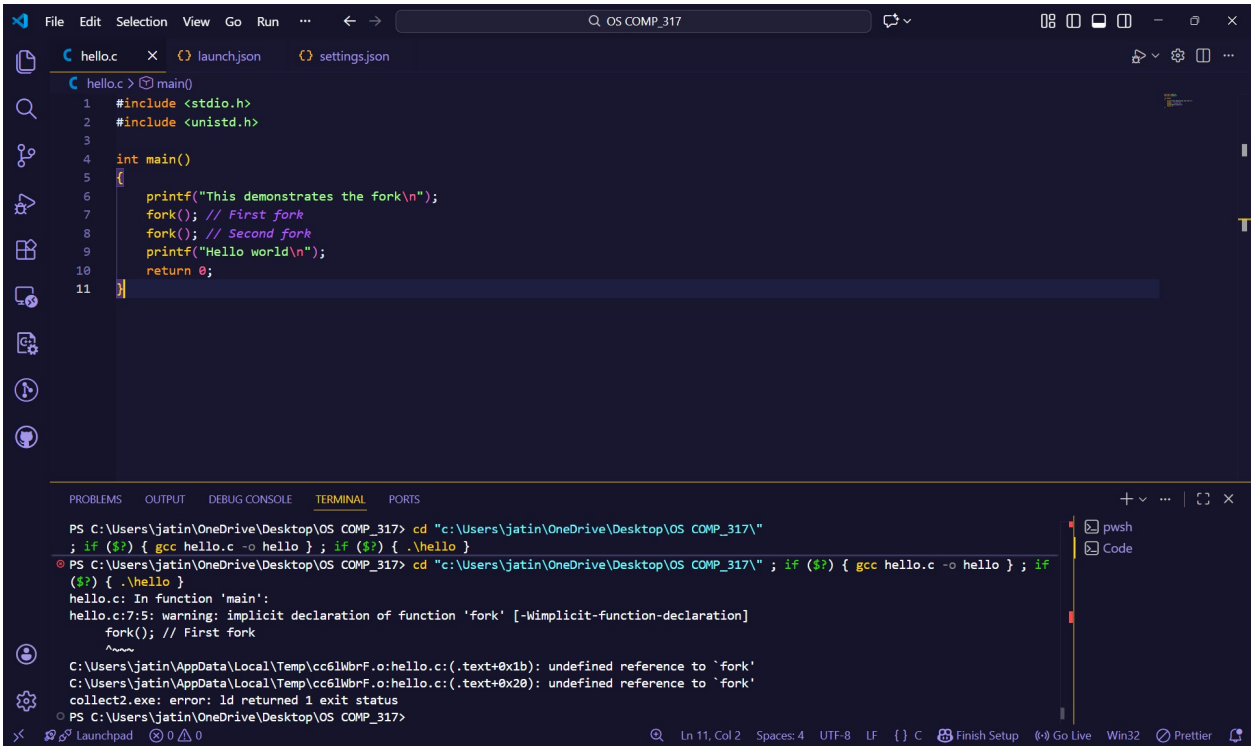
The `fork()` system call is a fundamental operation in Unix-like operating systems used for process creation. When a process calls `fork()`, the operating system creates a new process (the child) that is an exact duplicate of the calling process (the parent). Both processes continue execution from the instruction immediately following the fork call, but they possess unique Process IDs (PIDs).

Common uses of fork include:

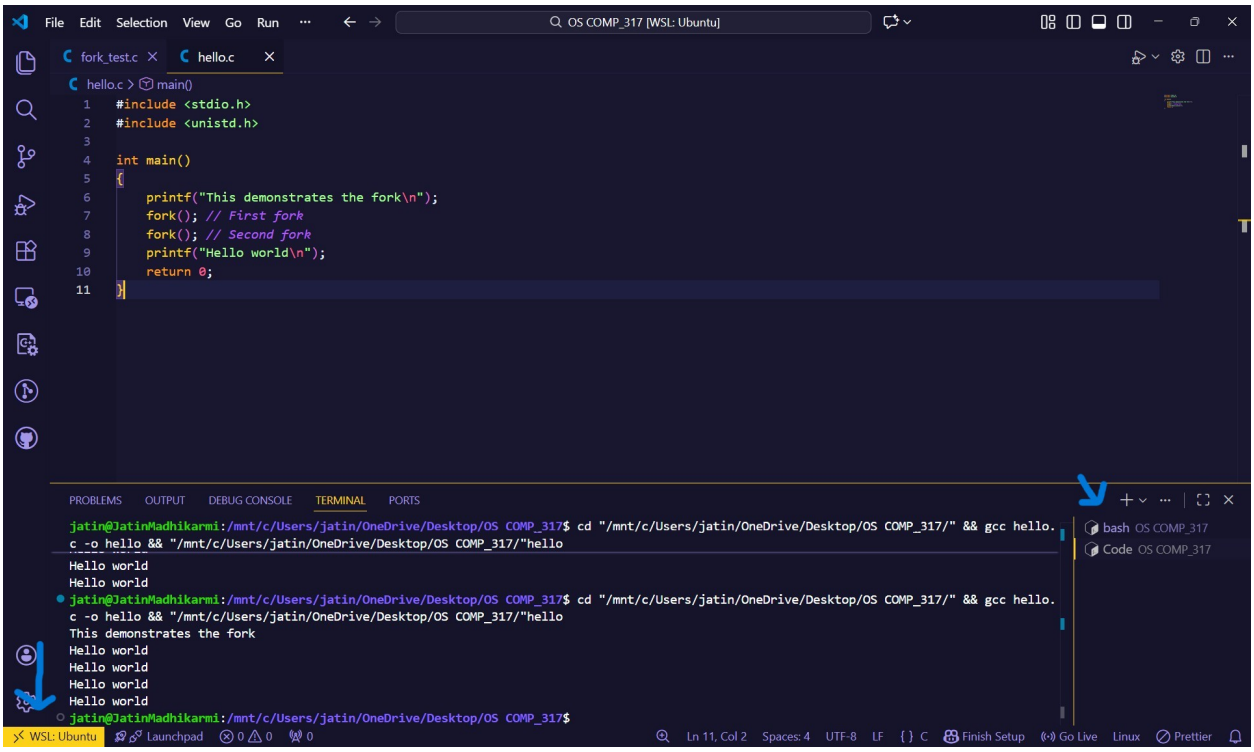
- **Parallel Processing:** Allowing a program to handle multiple tasks simultaneously.
- **Server Management:** Web servers use fork to create a new process to handle each incoming client request.
- **Shell Operations:** Shells use fork to execute user-entered commands in a separate environment.

Important Note

- **System Environment:** This program must be executed on a **Unix-based distribution** (such as Ubuntu, Debian, or macOS).
- **Windows Compatibility:** Standard Windows environments do not support the `unistd.h` header or the `fork()` system call natively. Please use WSL (Windows Subsystem for Linux) if a native Linux system is unavailable.



(a) Using the fork() in Windows System results in an error



(b) Using fork() in WSL system is sucessfully compiled

Program Code

The following C code demonstrates the creation of multiple processes using two consecutive `fork()` system calls.

```c
#include <stdio.h>
#include <unistd.h>

int main()
{
    printf("This demonstrates the fork\n");
    fork(); // First fork
    fork(); // Second fork
    printf("Hello world\n");
    return 0;
}
```

(a) Souce Code for implementation of two fork()

```
jatin@JatinMadhikarmi:/mnt/c/Users/jatin/OneDrive/Desktop/OS COMP_317$ cd "/mnt/c/Users/jatin/OneDrive/Desktop/OS COMP_317/" && gcc hello.
c -o hello && "/mnt/c/Users/jatin/OneDrive/Desktop/OS COMP_317/"hello
This demonstrates the fork
Hello world
Hello world
Hello world
Hello world
```

(b) Output for implementation of two fork()

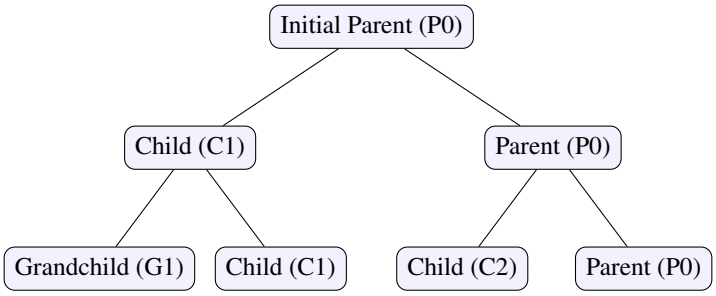
Experimental Results

When the program is executed, the string **"Hello world"** is printed **4 times**.

- The initial string ("This demonstrates...") prints once because it precedes the forks.
- Each `fork()` doubles the number of active processes.

Process Tree Diagram

The diagram below illustrates the hierarchy of processes created. Each node represents a process, and the edges represent a `fork()` operation.



Note: The leaf nodes represent the four independent processes that exist after two forks, each printing "Hello world".

Analysis

A `fork()` system call creates a duplicate of the current process. The new process (child) starts execution from the instruction immediately following the `fork()`.

For n consecutive fork calls, the total number of processes generated is 2^n. In this experiment, $n = 2$, resulting in $2^2 = 4$ processes. Each process maintains its own address space but continues execution from the same program counter, leading to four distinct "Hello world" outputs.

2 Modified Program Code (Three Forks)

The following C code demonstrates the creation of multiple processes using three consecutive `fork()` system calls.

```c
#include <stdio.h>
#include <unistd.h>

int main()
{
    printf("This demonstrates the fork\n");
    fork();
    fork();
    fork();
    printf("Hello world\n");
    return 0;
}
```

(a) Souce Code for implementation of three fork()

```
jatin@JatinMadhikarmi:/mnt/c/Users/jatin/OneDrive/Desktop/OS COMP_317/Programs$ cd "/mnt/c/Users/jatin/OneDrive/Desktop/OS COMP_317/Progra
ms/" && gcc fork*3.c -o fork*3 && "/mnt/c/Users/jatin/OneDrive/Desktop/OS COMP_317/Programs/"fork*3
This demonstrates the fork
Hello world
Hello world
Hello world
Hello world
Hello world
Hello world
Hello world
Hello world
```

(b) Output for implementation of three fork()

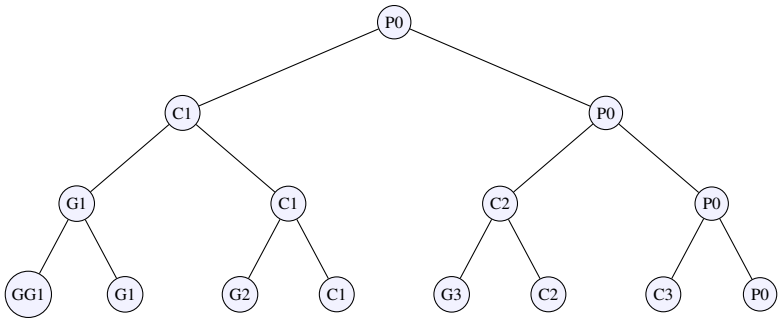
Experimental Results

When the program is executed, the string **"Hello world"** is printed **8 times**.

- The initial string ("This demonstrates...") prints exactly once.
- The final total number of processes created is $2^3 = 8$.

Process Tree Diagram

The diagram below shows how the single parent process branches out after three consecutive fork calls. Each level represents a `fork()` execution.



Note: P=Parent, C=Child, G=Grandchild, GG=Great-Grandchild. The 8 leaf nodes at the bottom represent the 8 processes that print "Hello world".

Analysis

The exponential increase in the number of "Hello world" prints is due to how the operating system handles the execution state during a `fork()` call.

Duplicate Execution State

When `fork()` is called, the child process is not a "fresh" start of the program; instead, it inherits the parent's current **Program Counter**. Consequently, both the parent and the child continue execution from the instruction immediately following the `fork()` line.

The 2^n Growth

Each subsequent `fork()` call doubles the number of existing processes:

1. **Before Forks:** 1 process exists.
2. **After 1st Fork:** 2 processes (2^1) exist and move to the next line.
3. **After 2nd Fork:** The 2 existing processes both fork, creating 4 processes (2^2).
4. **After 3rd Fork:** The 4 existing processes all fork, resulting in **8 total processes** (2^3).

Since each of these 8 processes eventually reaches the final `printf("Hello world\n");` statement, the output appears eight times in the terminal.

3 Process Differentiation using Return Values

Objective

To demonstrate the use of `fork()` return values to differentiate between parent and child execution paths and to understand process identification.

Program Code

```c
#include <stdio.h>
#include <unistd.h>
int main()
{
    int pid;
    printf("I am the parent process with ID %d\n", getpid());
    printf("Here I am before the use of forking\n");
    pid = fork();
    printf("Here I am just after forking\n");
    if (pid == 0)
        printf("I am a child process with ID %d\n", getpid());
    else
        printf("I am the parent process with PID %d\n", getpid());
    return 0;
}
```

(a) Source Code for understanding fork() with the PID

```
jatin@JatinMadhikarmi:/mnt/c/Users/jatin/OneDrive/Desktop/OS COMP_317/Programs$ cd "/mnt/c/Users/jatin/OneDrive/Desktop/OS COMP_317/Progra
ms/" && gcc fork_advanced.c -o fork_advanced && "/mnt/c/Users/jatin/OneDrive/Desktop/OS COMP_317/Programs/"fork_advanced
I am the parent process with ID 13514
Here I am before the use of forking
Here I am just after forking
I am the parent process with PID 13514
Here I am just after forking
I am a child process with ID 13515
```

(b) Output for understanding fork() with the PID

3.1 Discussion and Questions

1. Difference between pid == 0 and pid $>$ 0 In a Unix system, `fork()` returns a `pid_t` value that acts as a signal to the code:

- **pid == 0:** This corresponds to the **Child Process**. The OS returns 0 to the child to indicate it was successfully created.
- **pid $>$ 0:** This corresponds to the **Parent Process**. The value returned is the actual Process ID (PID) of the newly created child.

2. Differing PID Prints The parent and child print different PIDs because they are distinct entries in the Operating System's process table. Even though the child is a clone of the parent, it is assigned a unique PID by the kernel for tracking, scheduling, and security purposes. When each calls `getpid()`, the kernel retrieves the specific ID associated with that unique process context.

3. Output Order and Determinism While the parent lines often appear first, the order is **not guaranteed** and can change.

- **Why it can change:** Once `fork()` is called, there are two independent processes. The order in which they execute depends entirely on the **OS Scheduler**.
- **CPU Scheduling:** If the CPU is busy or if the scheduler decides the child has a higher priority at that millisecond, the child's output will appear first. This unpredictable behavior is a classic example of concurrency.

Conclusion

Through these experiments, we observed that `fork()` creates a new process by duplicating the existing one. We successfully verified the 2^n growth rule for multiple forks and demonstrated how a parent process can use return values to delegate specific tasks to a child process.